Клеточный автомат простыми словами

Окрестность фон Неймана и разбор пакета `automation`

Разбор проекта лабораторной работы, вариант 2

Идея пособия. Понять, что происходит с клетками на одном шаге, как пять клеток превращаются в номер от 0 до 31 и как этот номер используется в Go-коде.

1. Что такое клеточный автомат

Клеточный автомат — это сетка клеток, которая меняется шаг за шагом. Каждая клетка имеет одно из заранее заданных состояний. В нашей программе состояний два: клетка либо живая, либо мёртвая.

Состояние клетки на следующем шаге зависит не от всего поля сразу, а только от небольшой группы клеток рядом с ней. Это правило применяется одинаково ко всем клеткам поля.

Можно представить поле как фотографию. Чтобы построить следующую фотографию, программа смотрит на старое поле, вычисляет новое состояние каждой клетки, записывает результаты отдельно и только потом заменяет старое поле новым. Поэтому все клетки обновляются одновременно: результат для одной клетки не влияет на расчёт другой клетки в том же шаге.

В коде состояние хранится как значение типа `bool`: `true` — живая клетка, `false` — мёртвая.

2. Окрестность фон Неймана

Окрестность — это клетки, чьи состояния учитываются при обновлении одной выбранной клетки. В окрестности фон Неймана берутся четыре клетки, соприкасающиеся с центральной общей стороной: сверху, снизу, слева и справа. В нашем правиле учитывается также сама центральная клетка, поэтому всего получается пять значений.

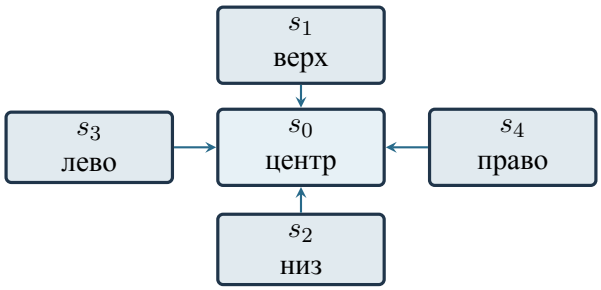


Рисунок 1 — клетки, состояния которых используются для обновления центра

Диагональные клетки в эту окрестность не входят. Это отличает её, например, от окрестности Мура, где учитываются также четыре диагонали.

В программе значения идут в строго заданном порядке:

центр, верх, низ, лево, право.

Важно не менять этот порядок только в одном месте: по нему и формируется номер конфигурации, и выбирается результат из правила.

3. Как пять состояний превращаются в номер

У каждой из пяти клеток два возможных состояния: 0 или 1. Поэтому всего есть $2^5 = 32$ разных комбинации. Каждой комбинации соответствует номер от 0 до 31.

Пусть a, b, c, d, e — состояния центра, верхней, нижней, левой и правой клеток соответственно. Номер комбинации — это просто двоичное число, составленное из этих пяти значений:

$$k = 16a + 8b + 4c + 2d + e.$$

Например, для набора $1, 0, 1, 0, 0$ получаем двоичную запись 10100_2, а значит номер $k = 20$.

В `field.go` этот расчёт выполняет `neighbourhoodIndex`:

```
bits := [5]bool{
    f.At(row, col),    // центр
    f.At(row-1, col), // верх
    f.At(row+1, col), // низ
    f.At(row, col-1), // лево
    f.At(row, col+1), // право
}
index := 0
for _, bit := range bits {
    index <<= 1
    if bit {
        index++
    }
}
```

Переменная `bits` содержит пять состояний. Цикл каждый раз сдвигает накопленный номер влево на один разряд и, если очередное состояние равно `true`, добавляет единицу.

Для набора $1, 0, 1, 0, 0$ это выглядит так:

Прочитано	1	0	1	0	0
Номер после шага	1	2	5	10	20

То есть цикл не ищет комбинацию в списке. Он сам собирает её номер из битов.

4. Что хранится в `Rule`

В Go нет классов в привычном смысле, здесь используются структуры и методы. Структура `Rule` хранит номер правила:

```
type Rule struct {
    Number uint32
}
```

Тип `uint32` означает беззнаковое целое из 32 бит. Поскольку у пяти клеток 32 комбинации, каждый бит числа хранит ответ для одной комбинации: 0 — центр должен стать мёртвым, 1 — живым.

Для варианта 2 в проекте используется формула:

$$R = 2 \cdot 11 \cdot 2006 \cdot 13 \cdot 12 = 6\,884\,592 = 00000000011010010000110011110000_2.$$

Нули слева добавлены, чтобы запись содержала ровно 32 бита. В методе `Bits` формат `%032b` как раз означает: вывести число в двоичной системе и дополнить его нулями слева до 32 символов.

Метод `transition` проверяет нужный бит:

```
func (r Rule) transition(configuration int) bool {
    return r.Number&(1<<uint(31-configuration)) != 0
}
```

Здесь `configuration` — номер комбинации от 0 до 31. Самый первый набор, `00000`, соответствует старшему (самому левому) биту. Поэтому для перехода к номеру k код сдвигает единицу на $31 - k$ позиций, оставляет только этот бит операцией `&` и проверяет, не равен ли результат нулю.

Пример. Набор 10100 имеет номер 20. Для него проверяется бит с номером $31 - 20 = 11$ при счёте справа с нуля. В правиле варианта 2 этот бит равен 1, поэтому новая центральная клетка будет живой.

5. Что хранится в `Field`

`Field` хранит всё текущее поле и правило, по которому оно меняется:

```
type Field struct {
    cells [][]bool
    rule  Rule
}
```

- `cells` — квадратная матрица состояний. Первый индекс — строка, второй — столбец.
- `rule` — копия правила, которое применяется при каждом шаге.

Основные методы поля:

- `New(size, rule)` создаёт квадратное поле. Новые значения `bool` изначально равны `false`, значит все клетки мёртвые.
- `Size()` возвращает размер стороны поля.
- `At(row, col)` читает состояние клетки.
- `Set(row, col, state)` задаёт клетке состояние.
- `Toggle(row, col)` меняет состояние на противоположное.
- `Clear()` делает все клетки мёртвыми.
- `Randomize(seed)` случайно заполняет поле. Одинаковый `seed` даёт одинаковую последовательность случайных состояний.

Чтение и изменение клетки проходят через `wrap`. Эта функция реализует тороидальные границы: если выйти за край, координата появляется с другой стороны поля. При размере 5 координата -1 превращается в 4, а координата 5 — в 0. В коде используется формула

$$\text{wrap}(x) = ((x\%N) + N)\%N,$$

где N — размер поля. Добавление N нужно, чтобы результат был неотрицательным и оставался допустимым индексом от 0 до $N - 1$.

6. Как работает один шаг

Главный метод обновления — `Step`. Он проходит по каждой клетке, находит номер её окрестности, получает ответ из `Rule` и записывает его во временное поле:

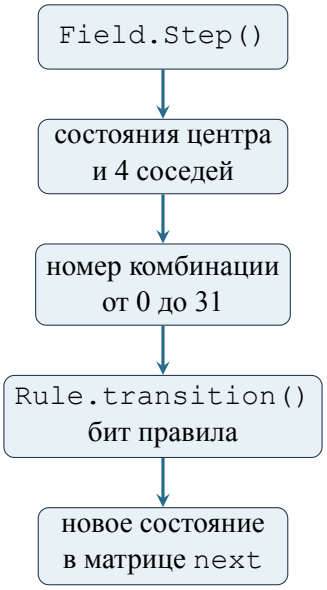
```
func (f *Field) Step() {
    next := makeCells(f.Size())
    for row := range f.cells {
        for col := range f.cells[row] {
            next[row][col] = f.rule.transition(
                f.neighbourhoodIndex(row, col),
            )
        }
    }
    f.cells = next
}
```

По порядку здесь происходит следующее:

1. `makeCells` создаёт пустую матрицу для нового поколения.
2. Вложенные циклы перебирают все строки и столбцы.
3. `neighbourhoodIndex` читает центр и четыре соседние клетки, затем превращает их состояния в число 0–31.
4. `transition` проверяет соответствующий бит правила.
5. Результат записывается в `next`, а не в старое поле.
6. Когда все клетки рассчитаны, строка `f.cells = next` заменяет старое поколение новым.

Именно временная матрица обеспечивает одновременное обновление: каждая клетка нового поколения рассчитывается по одному и тому же старому полю. Для поля $N \times N$ один вызов `Step` обрабатывает N^2 клеток, поэтому время работы растёт пропорционально площади поля.

7. Вся цепочка в одном месте



Коротко: **смотрим на пять клеток → кодируем их нулями и единицами → получаем номер → читаем бит правила → записываем новое состояние**.

Материал составлен по текущему коду проекта в `lab1/internal/automation`.